Para el SKIPCOND existen 4 condicionales:

- 000: Skip si AC < 0 (Si es negativo, menor a 0)
- 400: Skip si AC = 0
- 800: Skip si AC > 0 (si es positivo o 1)
- 0C00: Skip si AC ≠ 0 (si AC es diferente de 0, mientras no sea 0 continua, puede ser tanto negativo como positivo)

Para una multiplicación en MARIE no existen operadores para multiplicar entonces lo que se hace es sumar A un total de B veces dentro de un bucle
Inicio, LOAD B / Carga el multiplicador
SUBT Uno / Resta 1 (vamos a usar B como contador)
SKIPCOND 000 / Si B < 0 (es decir, el contador llegó a -1), termina
JUMP Multiplicar
JUMP Mostrar

Multiplicar, STORE B / Actualiza el contador B
LOAD Resultado
ADD A / Suma A al Resultado
STORE Resultado
JUMP Inicio / Repite el bucle

Mostrar, LOAD Resultado
OUTPUT / Muestra 12
HALT

/ --- DATOS ---
A, DEC 4 / Número a multiplicar
B, DEC 3 / Multiplicador (contador)
Resultado, DEC 0
Uno, DEC 1

Punteros: Guardar y Modificar Direcciones de Memoria
MARIE puede acceder a posiciones dinámicas de memoria (como recorrer un arreglo), haciendo uso de LOADI y STOREI

- LOADI caja: Lee la dirección almacenada dentro de caja y trae a AC el dato de esa ubicación.
- Para continuar con la siguiente dirección de memoria se usa solo LOAD caja por ejemplo y se le suma 1:
  / --- Leer un dato usando un puntero y avanzar ---

LOADI Ptr / Lee el contenido almacenado en la dirección 100
OUTPUT / Imprime ese dato

```
      / Avanzar la dirección del puntero
      LOAD Ptr
      ADD Uno        / Ptr = Ptr + 1
      STORE Ptr      / Ahora Ptr apunta a la dirección 101
      HALT
```

/ --- DATOS ---
Ptr, HEX 100 / Variable puntero que guarda una dirección (100)
Uno, DEC 1

Como aquí, Ptr es la dirección de memoria y se necesita uno para que se pueda mover a la dirección 101 y terminar el programa.

Existe algo llamado ***JNS (Jump and Store Instruction Pointer)*** que es lo que nos ayuda a acceder a la memoria en lo que podemos decir como vector o matriz, o saltar a la dirección X para después ir a la X+1, esta ayuda bastante para crear funciones bloques de código dentro de MARIE como lo hicimos en este caso:

```
/ --- PROGRAMA PRINCIPAL ---
Inicio,     INPUT            / Lee un número
            STORE Num        / Lo guarda en la variable Num

            JNS Duplicar     / Llama a la subrutina "Duplicar"

/ --- VUELVE AQUÍ TRAS EL JNS ---
            OUTPUT           / Imprime el resultado que dejó el AC
            HALT             / Fin del programa principal

/ ========================================
/ SUBRUTINA: Duplicar
/ Multiplica el valor de 'Num' por 2
/ ========================================
Duplicar,   HEX 0            / Reserva la dirección de retorno
            LOAD Num
            ADD Num          / AC = Num + Num
            JUMPI Duplicar   / Regresa al programa principal
/ ========================================

/ --- DATOS DEL PROGRAMA ---
Num,        DEC 0
```

Para analizar strings:

```
Inicio,   LOADI Ptr      / Carga el carácter apuntado actualmente
          SKIPCOND 400   / ¿Es igual a 0 (Fin de cadena)?
          JUMP Contar    / No es 0: cuenta el carácter y avanza
          JUMP Fin       / Es 0: termina la lectura

Contar,   LOAD Longitud
          ADD Uno        / Longitud = Longitud + 1
          STORE Longitud

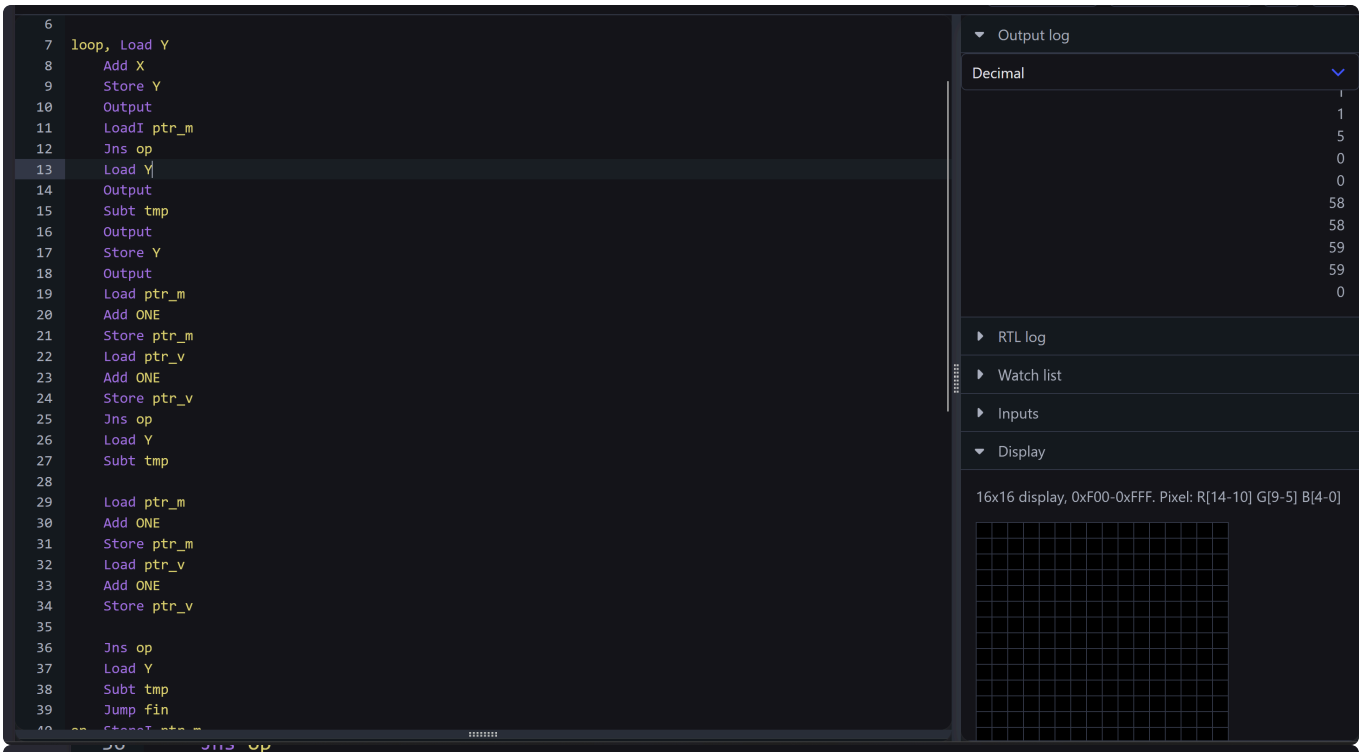
          LOAD Ptr
          ADD Uno        / Ptr = Ptr + 1 (Avanza a la siguiente letra)
          STORE Ptr
          JUMP Inicio    / Revisa la siguiente posición

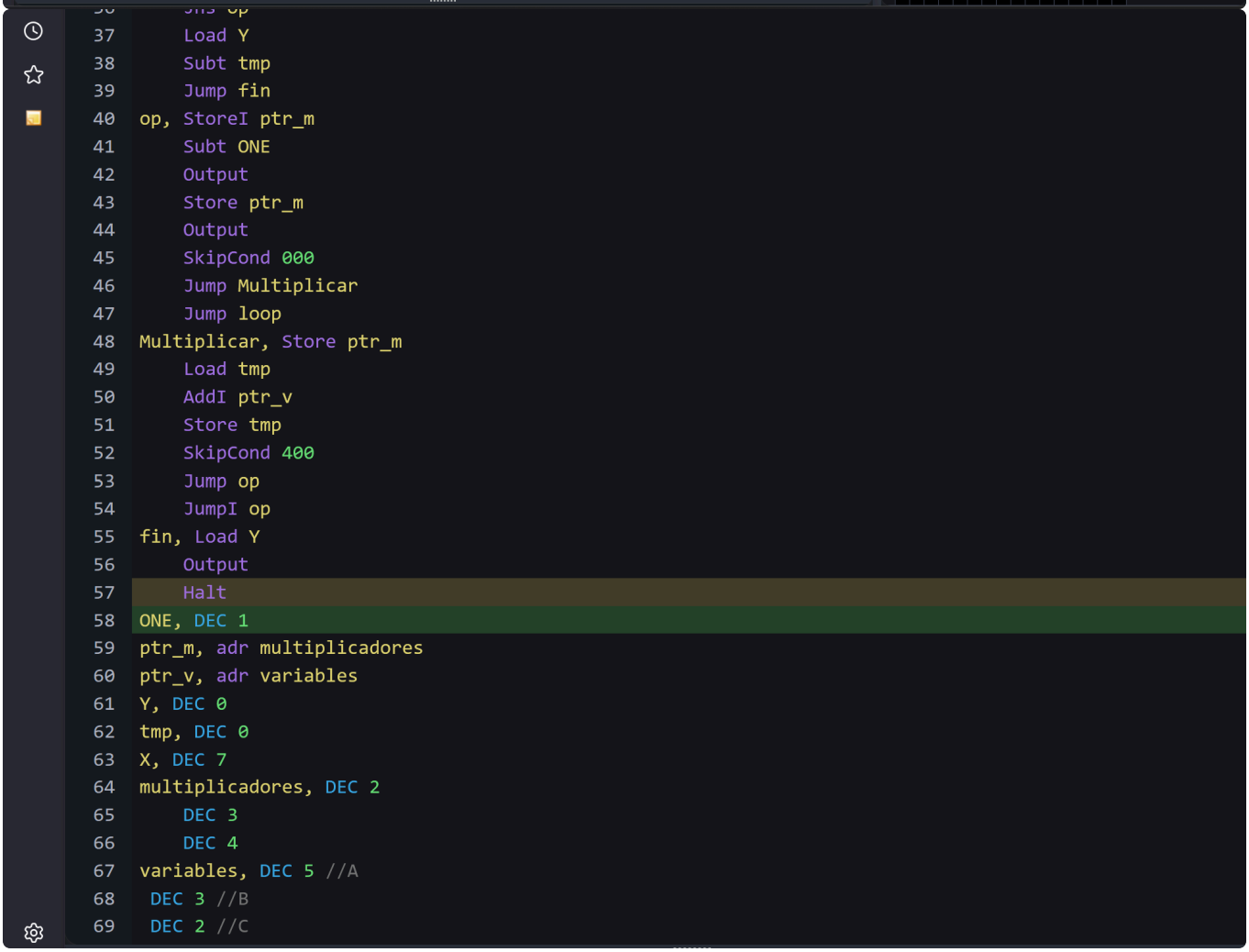
Fin,      LOAD Longitud
          OUTPUT         / Muestra la longitud total del String
          HALT

/ --- DATOS ---
Ptr,      HEX 200        / Dirección inicial de la cadena
Longitud, DEC 0
Uno,      DEC 1

/ --- CADENA EN MEMORIA (Ubicada en la dirección HEX 200) ---
/ Guarda "HOLA" en ASCII + Nulo
ORG 200
DEC 72                   / 'H'
DEC 79                   / 'O'
DEC 76                   / 'L'
DEC 65                   / 'A'
DEC 0                    / Termina la cadena (Null Terminator)
```

Basicamente la estructura para acceder a varias posiciones de memoria primero se debe de hacer una rutina de Inicio la cual siempre accedera a punteros de memoria osea LoadI ptr, despues se evalua con Skipcond y abajo de este se lleva a la rutina que se cumple mientras el ciclo se cumpla y si no salta automaticamente a finish, finish es la otra subrutina en donde es recomendable poner el output y halt





LA DIFERENCIA CLAVE ENTRE DIRECTO E INDIRECTO ES:

- Directo: Load x / Store x, opera directamente con el contenido guardado en la variable x, si x = 2 entonces en el AC se mandará un 2.
- Indirecto: LoadI x / StoreI x, Usa la etiqueta x como un puntero (Osea direccion de memoria). No da el valor o dato dentro de x, si no la direccion escrita en X.
  Explicado por Gemini con ejemplo:

Code snippet

```
Ptr,   HEX 200    / La celda 'Ptr' contiene la dirección 200
ORG 200
Dato,  DEC 50     / La dirección 200 contiene el valor 50
```

Instrucción	¿Qué hace?	Valor que termina en el Acumulador (`AC`) o Memoria
`LOAD Ptr`	Carga el valor contenido en `Ptr` .	`AC = HEX 200` (Carga la dirección misma).
`LOADI Ptr`	Lee el contenido de `Ptr` (`HEX 200`), va a la celda `200` y carga su valor.	`AC = DEC 50` (Carga el dato apuntado).

Load carga la direccion mientras que LoadI va a la direccion y muestra el valor que guarda esa direccion.